Context Engineering Frameworks for High-Fidelity Enterprise Software Agents: A Literature Survey

Hana Shalaby[1], Hoda Hussein[1], Mohamed El-Refai[1], Haya Shalaby[1], Laila Sayed[1], and Farida Elgharabawy[1]

***Abstract*—Amidst the rise of AI agents, their integration into the workforce has been swift and essential. Consequently, enterprises are starting to observe the limitations of their AI-supported and AI-powered systems, unreliability being one of the most concerning due to its hazardous operational consequences. One direction that has been particularly evaluated and undertaken is the implementation of context engineering techniques. In this literature review, we define context engineering, its advantages over prompt engineering and other model optimizations, its placement within Large Language Models (LLMs), and most notably, we present a thorough taxonomy and analysis of context engineering approaches in scholarship and industry, as well as compare and contrast them in terms of accuracy, efficiency, and usability in different applications.**

***Index terms*—Large Language Models, Context Engineering, Transformers, Software Agents, In-Context Learning, Agentic Workflows, Hybrid Retrieval Strategies**

I. INTRODUCTION

In 2026, the AI industry has changed drastically from "Prompt Engineering" (the optimization of natural language instructions) to "Context Engineering" where instead of focusing on the question, we focus on managing all the background information the AI sees. While LLMs have achieved impressive reasoning capabilities through scaling, their transition to enterprise software agents has introduced significant reliability challenges. In environments like Valeo, where agents must operate over massive codebases and complex frameworks, the issue is no longer raw model intelligence but the relevance of the information inside the context window.

The main challenge in building agents like this is the "Context Management Problem". This happens when AI's performance drops because the important details it needs are buried under "noise" or unimportant data [1]. While modern models can hold more data than ever, they still get confused by noise. This is why at Valeo the priority has shifted as the real challenge isn't just about the AI's raw intelligence, but about ensuring the massive amount of code it sees is relevant and clean.

This survey looks into the current LLMs architectures, agentic workflows, and hybrid retrieval strategies. Section II explores the architectures and scaling laws of LLMs. Then sections III and IV compares between consumer and enterprise use cases and the emergence of agentic workflows. After that, sections V and VI talk about specific context engineering techniques like pruning and injection and analyze their common failure modes. Section VII talks about hybrid retrieval and caching, followed by an evaluation of current industry products in section VIII. Together these sections give us the guidelines for developing a Context Optimization Engine designed to maximize reasoning accuracy in complex software repositories.

II. LARGE LANGUAGE MODELS (LLMS)

A. Foundations of Transformer Architecture

The improvement in LLMs today was because of the introduction of the Transformer architecture, which replaced classic and convolutional networks with an attention-based mechanism [2]. The traditional recurrent models processed data sequentially creating an issue that affected parallelization while the transformer uses a self attention mechanism to link inputs and outputs regardless of their distance in a sequence [2].

Attention based mechanisms function by calculating the weight of every token in a sequence relative to all other tokens simultaneously and it involves 3 vectors (Query (Q), Key (K), and Value (V)) [2], for every token the model calculates a relevance score by taking the dot product of its Query vector with the Key vectors of all other tokens which then is normalized to create "Attention distribution" [2]. Mathematically, this is expressed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \qquad (1)$$

This allows the model to treat the entire sequence as a single global context avoiding the information loss over long distances seen in older "hidden state" networks [2]

By putting "attention weights", the model can identify which specific words or code structures are most relevant to the

This work was not supported by any organization.

[1]Shalaby, Hussein, El-Refai, Shalaby, Sayed, Elgharabawy are with Department of Computer Science and Engineering, The American University in Cairo, Cairo, Egypt , {shalaby04, Hodamagdy-hussein, mohamedref11, hayashalaby, lailasayed0000, farida.elgharabawy}@aucegypt.edu.

current token being processed. For example, when analyzing a variable in a complex function, the mechanism can "attend" to its original declaration even if they are separated by hundreds of lines of code. This fixes the issue for needing sequential memory updates which enables parallelization required for training at scale [2].

B. Scaling to 175 Billion Parameters

A major improvement in this field is the scaling of the transformer capacity to improve performance. As shown by GPT-3, an LLM with 175 billion parameters representing a 10x increase over previous models enhancing task-agnostic capabilities [3]. Task-agnostic means the ability for the model to operate, learn, or make decisions across a wide array of diverse tasks without requiring specific architectural changes or task specific training data such that the model is not made for just one goal but instead it functions as a general purpose reasoner that identifies the desired goal through the instructions provided in the prompt.

The scaling of GPT-3 was not only achieved by parameter count but also from the diversity of its training data [3] as the model was trained on massive mixture of datasets which include filtered versions of Common Crawl, books, and Wikipedia [3]. These trillion of words of public human knowledge from all around the internet is what enables the "generalist" reasoning [3]. For the Valeo project, this tell us that public data provides broad intelligence but their limitation is that they still lack the high fidelity, private knowledge of the unique corporate codebase which justifies the need for a dedicated context optimization engine.

Research says that validation loss (the measure of a model's error) follows a smooth power law as a function of model size [3] which means that as parameters and data are increased, the model performance improves in a predictable, non linear mathematical relationship. This scaling is not just a matter of "more memory" as LLMs show a shift in in-context efficiency. Specifically, as the model scale increases, the "learning curve" within the prompt becomes steeper which means the model can recognize a task and reach high accuracy with significantly fewer examples (or "shots") than smaller models [3]. For enterprise grade agents, this means a massive model can understand complex coding standards at Valeo from just one or two snippets, whereas a smaller model might require dozens to achieve the same.

C. Instruction Tuning and Agentic Alignment

While scaling provides the "raw intelligence" for LLMs, intelligence alone does not guarantee a helpful agent. Early models were optimized for next token prediction, meaning they would often continue a prompt rather than answering it. To fix this, research introduced Reinforcement Learning from Human Feedback (RLHF) and Instruction Tuning [4].

Alignment training shifts the model's objective from simply "predicting the next word" to "following user intent." This is the specific advancement that allows an LLM to act as a software agent. Without this training, if a developer provides the prompt "Refactor this Python function," a raw model might simply provide more examples of unoptimized functions it found on the web. While an aligned model understands the command as an instruction to perform a specific task making it capable of software engineering workflows [4].

D. In-Context Learning and Meta-Learning

The transition to "meta-learning" represents a shift from models that simply memorize data to models that "learn how to learn." During pre-training phase, the model develops wide pattern recognition abilities by observing how humans solve problems across billions of pages of text [3]. This allows the model to act like a versatile student who has learned the "rules" of logic and communication, rather than a student who has only memorized specific answers for one subject.

These abilities are then used at inference time (when a user provides a prompt) to adapt to specific tasks through "in-context learning" [3]. In this process, the model uses its preexisting skills to recognize and follow a new pattern provided in the prompt (for example, learning a specific Valeo coding style from just two examples).

This adaptation occurs entirely within the forward pass of the model. This means that as the model reads the prompt from start to finish, it "simulates" learning for that specific moment without any weight updates. Unlike traditional AI training which requires a permanent mathematical change to the model's internal "brain," in-context learning does not change the model's permanent settings. The model stays exactly the same, using its general intelligence to solve a specific problem in the moment before "resetting" for the next conversation.

To evaluate these capabilities, research identifies a spectrum of task-specific data reliance [3]:

- Few-Shot (FS): The model is provided with a natural language description and as many demonstrations as fit in its context window (typically 10 to 100).
- One-Shot (1S): The model is given exactly one demonstration of the task.
- Zero-Shot (0S): The model receives only a natural language instruction without any examples.

E. Analysis and Insights

The primary insight from literature is that while zero-shot performance improves steadily with model size, few-shot performance increases more rapidly [3]. This suggests that larger models are better "meta-learners," able to absorb and apply the context provided in a prompt more effectively.

- Advantage: This allows humans and agents to easily switch between tasks, like switching from code generation to arithmetic [3].

- Gap: Despite these gains, models still struggle with natural language inference and "lost-in-the-middle" degradation where performance drops if the context is oversaturated with irrelevant data [1].
- Design Implication: For enterprise agents, the focus must shift to maximizing the quality of the "few-shot" context to ensure precision in complex reasoning tasks.

III. LLMS: INDIVIDUAL VS. ENTERPRISE-GRADE USE

A. Data and Structural Mismatch

Large language models are mainly trained on public, web data, and that training context strongly shapes both what they know and how they behave. When these models are deployed in enterprise environments, they are exposed to private datasets that differ substantially from their training data in both structure and semantics. Enterprise data is often fragmented across internal documentation, relational databases, and domain-specific schemas, where meaning is implicit and highly contextual. Empirical evaluations show that this mismatch leads to noticeable performance drops when LLMs are applied to enterprise tasks such as semantic annotation, text-to-SQL generation, or internal question answering, even when prompts are carefully constructed or include examples [5].

Recent studies evaluating GPT-style models on real enterprise databases suggest that many of these errors are not due to weak language understanding, but rather to incomplete or poorly aligned contextual grounding. Differences between public web data and enterprise data, both in content and structure, make it difficult for models to consistently map natural-language queries to internal representations [5]. In practice, this means that enterprise performance depends less on how capable the model is in general, and more on whether it is given internal context that actually reflects how the organization's data is structured and used.

B. Reliability Constraints and Enterprise-Specific Failures

When LLMs are deployed in enterprise settings, the most serious issues often appear not as obvious failures, but as subtle reliability problems. In particular, models may produce fluent but partially incorrect answers when operating over internal documentation or structured enterprise systems, especially when internal terminology or schema relationships are only partially captured in the provided context [5], [6]. This behavior is especially risky in enterprise workflows, where incorrect outputs may be given the green light and passed downstream into decision-making processes or automated pipelines.

A related issue is the mismatch between common evaluation benchmarks and real enterprise requirements. Public benchmarks typically emphasize recall and broad coverage across open-domain knowledge, while enterprise applications place a much higher priority on precision, traceability, and consistency under constrained or incomplete context. As a result, models that perform well on public benchmarks may still be unreliable in enterprise settings, where a single incorrect answer can carry significant operational consequences [7]. This makes it clear that improving enterprise reliability cannot be reduced to better prompts or higher benchmark scores alone.

C. Enterprise Question Answering and the Limits of Direct LLM Integration

Enterprise question answering highlights these challenges particularly clearly. Unlike consumer QA, enterprise QA focuses on domain-restricted queries over private company knowledge, where hallucinations are far less tolerable and more dangerous. Recent work on enterprise QA systems shows that retrieval-augmented generation (RAG) is often required to ground model outputs in internal knowledge bases; however, system performance depends heavily on retrieval quality, content organization, and evaluation strategy rather than model choice alone [7], [8].

Industry-focused studies emphasize that effective enterprise QA requires careful control over how context is assembled, including filtering, ranking, and validating retrieved information before it is passed to the model [8]. Taken together, this points to a missing layer between enterprise knowledge sources and the model, one that decides what context is appropriate and how it should be presented.

IV. HALLUCINATION IN LARGE LANGUAGE MODELS

Alansari and Luqman [9] present an extensive survey on hallucination in Large Language Models (LLMs), examining causes, detection methods, mitigation strategies, key findings, and future directions. They define hallucination as the fluent but factually incorrect or unsupported content generated by an LLM.

The authors describe the causes of hallucination in six main categories. The first one is data curation issues, which include inherent bias, imitative falsehoods, knowledge conflicts, domain deficiency, outdated knowledge, and long-tail knowledge gaps. The second one is model architecture, like attention mechanism limitations, positional encoding degradation, and unidirectional contextualization. The third cause of hallucination could be pre-training problems, such as shortcut learning, teacher forcing exposure bias, and insufficient negative examples. Fine-tuning also could cause hallucination due to overfitting and misalignment between model capabilities and alignment data expectations. The fifth reason is poor evaluation that suffers from inadequate metrics that assess fluency over factuality. The last cause of hallucination, according to Alansari and Luqman, is inference issues such as ambiguous prompts, sampling randomness, softmax bottleneck, and reasoning limitations in multi-hop tasks.

Next, they categorize detection methods into five approaches. The first suggested approach is retrieval-based methods, which compare outputs against external knowledge sources. They also suggest the implementation of uncertainty-based techniques that flag low-confidence outputs through token-level probability, semantic entropy, or supervised uncertainty heads. The third method is embedding-based approaches that measure semantic similarity between input and output using sentence embeddings, gradients, or spectral features. Learning-based methods that train supervised and unsupervised classifiers on annotated hallucination data were also suggested. Finally, the fifth approach is self-consistency methods that generate multiple responses and check for internal consistency across answers or paraphrased questions.

Then, Alansari and Luqman describe four categories of Mitigation strategies. The first is prompt-based techniques, which use templates, instructions, tags, or in-context learning prompts to guide factual generation. The second is retrieval-based methods that incorporate external knowledge before, during, or after generation through Retrieval-Augmented Generation (RAG) and knowledge graphs. The third category is reasoning-based approaches, which employ chain-of-thought, self-consistency, and iterative refinement to improve logical coherence. Finally, model-centric strategies involve decoding optimization through methods like knowledge distillation, supervised fine-tuning, and self-learning via contrastive and cross-lingual learning.

Finally, the authors highlight several critical needs for future research. These include adaptive attention mechanisms and better exploration strategies in reasoning, alongside stronger support for multilingual and low-resource languages. They also call for critique-based fine-tuning, granular benchmarks that go beyond binary labels, cross-lingual evaluation metrics, and standardized frameworks that balance detection accuracy with computational efficiency.

V. FROM PROMPT ENGINEERING TO CONTEXT ENGINEERING

A. Limits of Prompt-Centered Optimization

Prompt engineering has been widely used as a way to improve LLM outputs without changing the model itself. While this approach can be effective for small-scale or interactive use, it implicitly treats context as a transient, user-supplied artifact. Prompts provide no guarantees that the information included is complete, relevant, or trustworthy, nor do they offer mechanisms for separating instructions from untrusted data. As LLM applications move toward multi-step reasoning and tighter integration with external tools, these limitations become increasingly visible.

Empirical studies on long-context behavior further show that models do not reliably focus on relevant information as prompt length increases. Even when all required information is present somewhere in the input, performance can degrade if key details are buried within long or noisy contexts [1]. In other words, the problem is no longer how to phrase the prompt, but how to control what information the model actually sees.

B. Context Construction as a System-Level Problem

As these limitations became more apparent, recent work began to treat context construction as part of the system design rather than a prompting detail. Retrieval-augmented generation formalizes this shift by grounding model outputs in external knowledge sources instead of relying solely on parametric memory [10]. Subsequent surveys and enterprise-focused studies argue that retrieval quality, augmentation strategy, and robustness to incomplete or noisy context play a larger role in system performance than prompt design alone [11], [7].

This shift is particularly relevant in enterprise environments, where required knowledge is proprietary, frequently updated, and distributed across multiple systems. Instead of relying on static prompts, enterprise LLM applications must dynamically assemble task-specific context at reasoning time, often combining retrieved documents, structured query results, and system constraints. This shifts context from being an incidental input to something that must be deliberately designed and maintained.

C. Security, Governance, and Context Control

When context is assembled dynamically from multiple sources, new risks start to emerge. When LLMs consume information from retrieval pipelines, documents, or external tools, the boundary between instructions and data becomes less clear. Recent security analyses identify prompt injection and malicious context manipulation as major threats in LLM-based systems, particularly when retrieval or tool use is involved [12], [13]. These risks are amplified in enterprise settings, where inputs may originate from partially trusted or poorly validated internal sources.

At the same time, regulatory and governance considerations further restrict enterprise LLM design. Privacy and compliance requirements need clear provenance, access control, and auditability for any data incorporated into model context [14]. For enterprise deployments, this makes a controlled context mediation layer a practical requirement rather than an optional design choice.

VI. AGENTIC WORKFLOWS: ARCHITECTURES, PARADIGMS, AND QUALITY CONTROL MECHANISMS

A. Introduction to Agentic Workflows

An AI agent is a system that uses a large language model (LLM) as its core reasoning engine to determine the control

flow of an application while interacting with its environment to achieve a target objective. [15] Unlike traditional standalone LLM usage, where responses are generated from static pre-trained knowledge, AI agents enable dynamic interaction with external systems. They can retrieve real-time information, invoke tools such as calculators, databases, and APIs, and apply structured reasoning strategies to solve complex tasks.[16]

An agentic workflow refers to the structured process governing how such agents operate. It defines how tasks are decomposed, how information is retrieved, how decisions are made, and how outputs are validated. This paradigm represents a significant advancement in building sophisticated LLM-based applications. [16]

B. Emergence of Agentic Workflows

As scaling-based improvements in LLMs begin to plateau, research attention has shifted toward enhancing inference-time reasoning. Structured techniques such as chain-of-thought prompting, ReAct (reasoning and acting), and reflection have demonstrated substantial performance gains across reasoning, coding, and educational benchmarks. These techniques are foundational components of agentic workflows, enabling multi-step reasoning and tool utilization.

Emerging evaluation frameworks further emphasize the necessity of agentic systems for addressing complex, real-world tasks that extend beyond single-pass inference. [16]

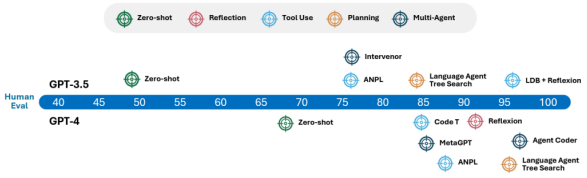


Figure 1: Performance of GPT-3.5 and GPT-4 (zero-shot) on HumanEval benchmark alongside agentic frameworks built on top of base GPT models. Agentic workflows demonstrate significant performance improvements over standalone zero-shot inference, with GPT-3.5-based agentic systems approaching or exceeding GPT-4 zero-shot performance. [16]

C. Core Structure of Agentic Workflows

A general agentic workflow typically follows three stages:

1) Planning stage: To reduce hallucination and to help in making tasks easier, the AI agent divides the the given objective into smaller tasks and execute them in sequence.
2) Information Retrieval and Tool Use: The agent retrieves all relevant information anf if not enough it combines internal and external tools to help collect needed data (e.g., web search, APIs, databases) to support task execution.
3) Execution and Refinement: The agent executes tasks, checks outcomes, reflects on intermediate results, and dynamically modify its strategy until the goal is achieved.[16]

Agentic behavior exists on a spectrum. Some systems incorporate minimal tool usage, while others involve complex multi-agent coordination. As research advances, agentic systems are expected to become increasingly autonomous and capable. [16], [17]

D. Paradigms of Agentic Workflows

Agentic workflows generally fall under four main paradigms:

1) Reflection: The agent asses its outputs and checks the steps taken to detect and correct errors.
2) Planning: The agent constructs and follows structured sub-tasks to achieve higher complex objectives.
3) Tool Use: The agent integrates external tools such as code execution engines, search systems, or APIs to be able to increase its ability in achieving the goal .
4) Multi-Agent Collaboration: Multiple specialized agents communicate and coordinate to solve a shared problem. [15], [17]

E. Motivation and Challenges

Recent advancements in program synthesis clarify strong LLM capabilities in software development tasks. Agentic workflows extend zero-shot generation by working on multiple LLM instances that helps in reflection, debate, and collaborate reasoning. This allows the integration of traditional software engineering practices such as automated testing and self-debugging into LLM-based systems.[18]

However, current approaches exhibit several limitations:

- Assumption of Visible Unit Test Conformity: Generated code is often assumed to align perfectly with visible unit tests.
- Bottleneck of Synthesized Test Quality: Automatically generated tests may contain errors, misleading the debugging process.
- Deviation of Self-Debugging Trajectory: Repeated debugging steps may stagnate, form loops, or introduce new errors. Many multi-agent workflows remain static and lack adaptive control mechanisms.[18]

F. Agentic Workflows vs. Autonomous Agents

Agentic workflows and autonomous agents represent distinct paradigms.

Agentic workflows rely on predefined structured pipelines composed of multiple coordinated LLM invocations. [15] Autonomous agents, in contrast, operate through dynamic environment-driven decision-making, adapting reasoning patterns in real time.[16]

Compared to autonomous agents, agentic workflows are more amenable to systematic optimization because they can be constructed from domain knowledge and iteratively refined. [15]

G. Types of Agentic Workflows

Agentic workflows may be categorized as:

- General-purpose: these workflows apply universal reasoning strategies across tasks
- Domain-specific: these ones are application specific such as: code generation, mathematics, or data analysis.

Although many workflows are manually designed, exhaustive domain-specific design is impractical, motivating automated workflow generation.[15]

H. Automated Agentic Optimization

We can optimiz

1) Prompt Optimization: Improving prompts within fixed workflow structures.
2) Hyperparameter Optimization: Tuning parameters such as temperature or model choice.
3) Workflow Structure Optimization: Automatically searching over workflow architectures.

While prompt and parameter tuning offer incremental improvements, structural optimization provides greater automation potential. Systems such as GPTSwarm and ADAS explore graph-based representations, but often face expressivity or efficiency limitations.

Code-based workflow representations offer greater flexibility by naturally supporting sequential logic, branching, loops, and hybrid structures. Techniques such as Monte Carlo Tree Search (MCTS) have been proposed to efficiently explore workflow search spaces using execution feedback.[15]

I. Formal Representation of Agentic Workflows

An agentic workflow can be conceptualized as a structured composition of LLM-invoking nodes connected through defined execution relationships.

Each node is parameterized by:

- The selected model,
- The input prompt,
- The temperature setting,
- The output format.

Edges determine execution logic. While graph representations support hierarchical and parallel execution, they require extensions to handle conditional logic. Neural network representations allow adaptive transitions but lack precise execution control.[15]

Code-based representations inherently support sequential execution, conditional branching, loops, and hybrid structures. Therefore, code provides the most expressive and controllable representation for agenetic workflows.

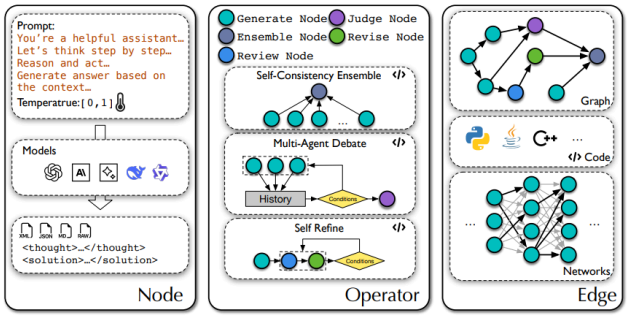


Figure 2: This is one of the referenced examples about node,operator and edge with their structured parameters.[15]

J. Multi-agent architecture

There are five types of architectures which uses multi-agents: single, Network, Supervisor, Hierarchal and Custom architectures. An example of this is Supervisor architecture which uses master agent to lead other agents with each one specified to perform a specific task. And Network architecture which allows each agent to communicate with another agent in the system.

Architecture	Key Characteristics	Potential Challenges
Single (Baseline)	- Utilizes a single LLM that can call multiple tools. - Represents the simplest setup.	- Subject to previously noted limitations such as overwhelmed context and insufficient specialization.
Network	- Composed of numerous agents, each with its own tools and the ability to call upon other agents. - Often employed by projects such as Swarm or Crew AI. - Lacks a designated "leader," allowing any agent to invoke any other agent at any time.	- Less predictable behavior. - Potentially expensive due to numerous LLM calls. - Time-consuming and complex to manage in production.
Supervisor	- Utilizes a single "supervisor" agent that decides which sub-agent to call next. - Sub-agents focus on specific domains, enhancing specialization. - A simplified variant treats sub-agents as tools used by the main agent, where the main agent's tool calls handle inter-agent communication.	- Over-reliance on the supervisor agent for optimal orchestration. - Potential bottleneck if the supervisor agent is overwhelmed.
Hierarchical	- Builds on the supervisor model by introducing multiple layers of supervisor agents. - Allows for large, domain-specific systems to be organized in tiers.	- Complexity increases with each additional supervisory layer. - May require extensive coordination across tiers.
Custom	- Tailored specifically to the requirements of a given domain. - Often combines elements from supervisor or hierarchical models with custom domain logic.	- Development can be resource-intensive due to bespoke design. - Requires ongoing maintenance for evolving domain needs.

Table 1: This table compares between the five multi-agent architecture strategies in terms of the characteristics of each one and the potential key challenges that faces this strategy. [16]

VII. EXISTING BENCHMARKS AND THEIR INSUFFICIENCY

Some of the benchmarks introduced for testing and evaluating LLM's problem solving capabilities are: MMLU [19],

Big-Bench [20], and AGIEval [21] MMLU: This benchmark evaluates a model's knowledge and problem-solving ability through multiple-choice questions starting from basic to difficult levels.[19] Big-Bench: This one evaluates language models on tasks believed to be highly more than the current capabilities or the model, including novel linguistic reasoning, common sense, mathematics and software development.[20] AGIEval: is a benchmark designed to help foundation models in the context of human-centric standardized exams, such as college entrance exams (SAT, GRE, GMAT). [21]

However these Benchmarks are still insufficient because of the following reasons:

1) Most of these benchmarks tests the model single response or interaction, not multiple operations that has to be executed sequentially to check a single result.
2) They test the Model's ability of using single static database not a dynamic changing one.
3) They don't test the model's external tool use

[19], [20], [21]

VIII. CONTEXT ENGINEERING

A. Introduction and Definitions

We begin by clarifying key terms. The integration of external knowledge into LLMs presents what we might call the "context management problem": how to provide models with sufficient relevant information without overwhelming them. Two main solutions have dominated discussion, though neither proves entirely satisfactory alone.

Retrieval Augmented Generation, or RAG, represents the more selective approach. The concept is straightforward: when presented with a query, retrieve relevant documents from an external corpus using similarity search, then feed these as context to the LLM for generation. This approach, pioneered by Lewis et al. [22], excels at precision but struggles with comprehensiveness. Most implementations use relatively short retrieval units, typically 100 to 512 tokens, to conserve limited context window space.

Long Context processing takes almost the opposite approach. Why retrieve when you can simply include everything? Advances in transformer architectures now allow processing of remarkably long sequences, from tens of thousands to millions of tokens. The appeal is obvious: comprehensive context without retrieval overhead. The reality, as we will see, proves more complex.

Between these extremes lie hybrid strategies. These systems attempt to capture RAG's precision while leveraging LC's comprehensiveness, typically through some form of staged processing. Recent studies demonstrates convincingly that such hybrids outperform pure approaches across diverse scenarios.

B. Context Pruning

a) *Context Pruning for Large Language Models:*

With the rise of longer prompts and the daily usage of LLMs for more complex natural language tasks, the importance of how we request tasks from our models has increased significantly [23], [24], [25]. The complexity of modern LLM tasks often manifests in larger contextual requirements, including detailed specifications, multiple edge cases, and large supporting documents. This leads to a rise in the number of in-context learning (ICL) examples in prompts and the details required to serve requirements [3]. This section of the paper aims to answer the question: "What efforts have been made to solve the problem of *what* information to include in the context provided to an LLM?"

To get the most out of an LLM, approaches for optimization can be categorized into model-centric and prompt-centric. Some model-centric methods include fine-tuning the model to a specific task using parameter pruning, while prompt-centric approaches involve taking the input itself and after tokenizing it, applying operations such as prompt compression and prompt design [23]. Context pruning, which is a prompt compression technique, is used at the input level. It differs from other prompt compression techniques which may rewrite or encode prompts to reduce length of the input sequence. This makes it particularly useful for creating engines that interact with LLMs as a black box [26].

Furthermore, prompt-centric optimization has risen in value as it has proven to reduce computational cost while maintaining results [25]. Findings from the paper on dynamic context pruning by Anagnostidis et al. demonstrate that pruning 80% of context without notable performance loss. Most modern LLMs are based on the Transformer architecture, which models dependencies between all token pairs through self-attention mechanisms. In the landmark paper "Attention Is All You Need," the computational cost of self-attention grows quadratically with the input sequence length, as each token attends to all other tokens in the sequence [2]. As shown in the paper, the per-layer complexity of self-attention is O($n^2 * d$), where n is the sequence length and d the representation dimension [2] (Section 4 and Table 1). While the quadratic complexity was a trade-off for better parallelization, it motivates exploration of solutions to reducing problem size (the sequence length n).

The main inhibitor that is inevitable with long prompts is redundancy of certain tokens. Studies have shown that models are tolerant to the removal of stale context, and thus motivate systematic pruning techniques [25], [26]. Additionally, theoretical studies in information theory posit that natural human language contains redundancy which LLMs may not need for context, and they may come to the same conclusions that humans do [27], [28]. Studies have shown that most words spoken by humans in natural language do not make a difference in the response of the LLM, or they already overlap with

what the model has learned, making them redundant. These are the target tokens that context pruning aims to reduce [29]. Experiments across four datasets done by Jiang et al. showcase performance competitive with state-of-the-art with up to 20x compression [26]. It is evident that context pruning, and subsequently reducing the number of input tokens, is essential to reducing computational costs later at the inference stage.

Furthermore, the context window is finite, and for chats that remain for long periods of time, older prompts become less relevant to the user's current needs. Context pruning is a solution to removing unnecessary tokens taking up space in the context window [29].

These observations motivate the need for systematic approaches to selecting and reducing contextual information prior to model inference. The following subsections review the principal methods proposed in the literature for achieving this goal.

While the terms context pruning and prompt compression are often used interchangeably in the literature, context pruning can be viewed as a specific subclass of prompt compression that focuses on selectively removing low-utility tokens rather than rewriting or encoding them. In this work, we adopt the term context pruning to emphasize the preservation of interpretability and faithfulness.

We define context pruning as the process of filtering or selecting a subset of the context such that prompt length is reduced while maintaining the task performance by retaining essential information [27]. Formally, context pruning aims to construct a reduced context $C' \subset C$ such that $|C'| < |C|$ while preserving task performance and faithfulness. It differs from other prompt-centric optimization techniques in the sense that it focuses on decreasing the length of the input sequence rather than, for instance, managing the location of certain tokens in the input sequence according to their importance [1].

b) *Taxonomy of Prompt Compression Methods:* It is important to be aware of the various prompt compression methods, as our project may employ a hybrid approach, which needs to be clearly defined. In the survey "Prompt compression", they provide a division of the various methods and label them under two classes, hard-prompt methods and soft-prompt methods[23]. The main difference between the two is that hard-prompt methods operate directly on the input tokens, not interacting with the model, while soft-prompt methods involve creating word embeddings of the prompts that are of the same size as the vector embeddings of the LLM's dictionary. This means we require model access for soft-prompt methods.

c) *Selective Context [23], [29]:* use the concept of self-information to recognize which terms are essential to the generation of a response. They model the information of a token as the log likelihood of the output probability. (insert the formula here and cite) Other features of information gathered about natural language include the entropy and perplexity. The sum of these values gives the self-information of a token, and then decisions on whether this token should be deleted or not are made based on the sum. Their methods are heavily reliant on the foundational papers done in the late 1940s and early 1950s on Information Theory, which study properties such as entropy, perplexity, and redundancy of linguistics. [28], [30] mainly

$$\begin{aligned} I(x_0, x_1) &= -\log_2 P(x_0, x_1) \\ &= -\log_2 P(x_0) P(x_1 \mid x_0) \\ &= -\log_2 P(x_0) - \log_2 P(x_1 \mid x_0) \\ &= I(x_0) + I(x_1 \mid x_0) \end{aligned} \tag{2}$$

Their method can be simplified into 3 steps:

1) Calculate the self information $I(x_i)$ of each token
2) Form *lexical units u* (phrases/sentences) with $u = (x_0, ...x_t)$ such that the self information of each unit is $I(u) = \sum_i I(x_i)$
3) Retain essential lexical units by sorting the lexical units by descending order of self-information, calculating the p-th percentile I_p of the self-information values, and obtain a filtered context C' as follows $C' = U_i \mid I(U_i) \geq I_p, 1 \leq i \leq n$

They conduct experiments on various models including GPT-4, LLaMa-7B, and Vicuna-13B. They test the models' performances of different tasks like question/answer, summarization and conversation by using the full context and the compressed context after applying Selective Context. Their results demonstrate that performance only significantly drops when the amount of context filtered out is high. They discuss the faithfulness of the models, which represents whether the model will complete the required task if it does not have sufficient information.

While the employed method shows promising results, the authors note that the percentile-filtering based method is not optimal as the performance varies for different tasks, giving rise to an optimization problem for finding the best percentile to maintain good performance across all LLM tasks [24], [29].

d) *Distillation Enhanced Filtering:*

LLMLingua and its successor LLMLingua-2 extend filtering-based prompt compression by incorporating data distillation to improve the quality and faithfulness of compressed prompts [26], [27]. While earlier filtering approaches relied primarily on token-level self-information metrics, LLMLingua-2 leverages a large teacher model (e.g., GPT-4) to generate distilled instruction–response pairs that serve as supervision for training a compression model. The authors formalize three objectives for prompt compression:

1) Compactness: the compressed prompt should be significantly shorter in length.
2) Informativeness: important task-relevant information must be retained.

3) Faithfulness – the compressed prompt must not introduce hallucinations or alter semantic meaning in a way that degrades downstream accuracy.

Importantly, although GPT-4 is used during the distillation stage, the resulting compression operates at the input-token level and does not require modification of the target LLM. Therefore, LLMLingua-2 falls under hard-prompt filtering methods, as it performs context reduction prior to inference without altering model parameters. This design makes it suitable for black-box LLM deployment scenarios, where access to model weights is unavailable and optimization must occur at the prompt level.

C. Context Injection

Context injection refers to the process of organizing, structuring, and inserting contextual information into an LLM prompt in a controlled and systematic manner to guide model behavior and reasoning. It focuses on how to include the remaining compressed prompts in the final request to the model. Studies have shown that positioning makes a difference between the emphasis placed on certain tokens [1]. This section of the paper aims to answer the question: "What efforts have been made to solve the problem of knowing *how* the chosen informations should be presented and structured?"

The study by Liu et al. discusses 2 forms of bias that LLM models tend to have, namely primacy bias and recency bias [1]. The former refers to the model's tendency to focus on information that was provided during the early stages of the chat, while the latter refers to the information that was provided during the recent stages of the chat. Indeed, performance degrades significantly when crucial information is given in the middle of the input context.

This motivates the study of positioning the information in a given prompt, to enforce that the most crucial aspects of a task are the ones given the most attention. The authors note a U-shaped performance curve for a task they name the "multi-document" question task. They performed experiments on the GPT3.5-Turbo model, and provide evidence that the performance was better when the prompt contained to external documents. They observe a trade-off between increasing context length to improve performance of the downstream task and the accuracy of the resulting query [1].

Anthropic [31] take on effective context engineering tells us the importance of separating roles within the prompts, differentiating between system level instructions, task-specific directive, and contextual data. Their guidelines focus on the clarity of boundaries between blocks and the prioritization of important instructions to fix these degradation effects. This includes using heuristics like XML tagging or Markdown headers to delineate sections.

Furthermore, context injection is increasingly seen as a compositional process rather than a flat buffer. For example Microsoft Research [32] supports the modular assembly of prompts where components like retrieved documents, reasoning traces and tool outputs are dynamically injected based on the execution state. In agentic systems, this usually involves a "just-in-time" approach, loading only light weight identifiers (like file paths or web links) to preserve the "attention budget" and avoid context pollution [31].

Despite these advantages, structures context injection faces some limitations like positional biases as they remain rooted in the transformer-based architectures, meaning structural heuristics cannot fully remove attention asymmetries [31] which is also shown by the U-shaped performance curves in [1]. Also, increasing the number of structured blocks can increase the total context length reintroducing costs that pruning methods intend to solve [1], [31]. Finally, there is currently no standardized way for prompt structuring making the evaluaition of injection strategies highly task dependent.

IX. HOW CONTEXT ENGINEERING FAILS

In theory, context engineering sounds like a good solution to problems when dealing with LLMs. It summarizes what is old and passes only the best and relevant pieces to the model. However, in reality the system can still fail to produce an efficient and sensible answer. The hard part comes when the system does not know what counts as the memory that should be used when the user needs it, and this is a gap between semantic relevance and task relevance. Retrieval systems like RAG are great when it comes to finding text that sounds similar to the user's new message or prompt. However, the user enters a prompt expecting an answer also relevant to another prompt entered 30 messages ago, which the LLM may or may not grasp. The system here can usually fail in retrieving the right memory, not because it is deleted or "forgotten", but because it doesn't see the relevance [33].

Another common reason that leads to failure after using context engineering is that summarization, while undoubtedly reduces noise, can unintentionally remove important details. The summaries often turn into more generic versions of the prompt. For example, a user stating that he hates technique A and would like to use framework B, may reach the LLM as the user would like to use framework B". This could then later cause frustration or a sense of misunderstanding if the LLM later suggests framework A again, because it was not told to eliminate it completely. This is a very simple example, but it shows how summaries don't just shorten the text but usually change the meaning. This drift can accumulate over time causing the system to be useless [33].

A third major issue when it comes to context engineering is conflicting memories. Real conversations are messy, as people could change their mind, speak hypothetically, and correct themselves. If everything is stored equally in the memory at the same time, it can later in the conversation retrieve

outdated preferences in the same way that it can retrieve recent preferences. So basically, the model gives an answer that is seemingly correct, but to an older version of the user. There should be a kind of ranking policy in the memory based on how recent it is and authority or importance to avoid the memory parts contradicting each other [33].

Context engineering also sometimes fails because of data that looks that it would be important to the system, but it is actually noise. This is because of emotionally intense phrases, cause by the nature of the human being, or numbers and locations for example, which are over-ranked by retrieval systems. The models could be seen suddenly fixating on this info that was never meant to be used [33].

Finally, in some cases even when the right and intended information is retrieved, the system can still fail in produces the intended output due to the way the retrieved content was formatted and sent to the model. The model can interpret a question for example as an instruction due to the way it was sent, leading to actions being taken falsely. So, the failure here isn't about the retrieval like the previous challenges, but how the retrieved content is structured in the prompt.

X. HYBRID RETRIEVAL STRATEGIES FOR LARGE LANGUAGE MODELS

A. Methodological Landscape

In this section, we examine six representative studies that together cover the key methodological dimensions of hybrid retrieval: how context is selected and structured, how context length affects model performance, how inference cost is reduced, and how models can learn to regulate their own retrieval behavior. Table I summarizes the key objective, approach, and findings of each study.

a) *Retrieval Augmented Generation (RAG) and Long-Context (LC) Approaches:*

Li et al. [34] conduct a comprehensive benchmarking study comparing Retrieval Augmented Generation (RAG) and Long-Context (LC) LLMs across nine datasets using three models: Gemini-1.5-Pro, GPT-4O, and GPT-3.5-Turbo, with two dense retrievers: Contriever and Dragon. LC consistently outperforms RAG across almost all settings: on average, LC surpasses RAG by 7.6% for Gemini-1.5-Pro, 13.1% for GPT-4O, and 3.6% for GPT-3.5-Turbo. However, there was a significant overlap between the performance of RAG and LC as they produced identical results for 63% of the queries. Using LC for the prompts where RAG gives an accurate answer is a wasted cost because LC uses drastically more tokens. This motivated the authors' approach of SELF-ROUTE. SELF-ROUTE is a two-step routing mechanism: in the first RAG-and-Route step, the LLM is given the retrieved chunks and prompted to either answer the query or declare it "unanswerable"; answerable queries use the RAG prediction as the final answer, while unanswerable queries route to the full LC pipeline. Across all three models, SELF-ROUTE routes more than half of queries to RAG, significantly reducing token usage. When it comes to performance, Across all three models, SELF-ROUTE surpasses RAG by over 5%, and show a slight drop for GPT-4O (-0.2%) and Gemini-1.5-Pro (-2.2%), but an improvement for GPT-3.5-Turbo (+1.7%). For RAG prediction, the authors divided long contexts into chunks of 300 words and studied the effects of the number of chunks fed to the LLMs (k) on the performance. For both RAG and SELF-ROUTE, a larger k improves the performance, making it closer to LC's performance. The advantage of SELF-ROUTE is the most significant for smaller k as with bigger K all three methods get similar performance. The authors mentioned the importance of finding the most optimal k when applying SELF-ROUTE to other applications.

LongRAG, from Jiang et al. [35], implements a hybrid approach where they increase the unit size but decrease the total number of units by grouping related documents into one unit. Four different datasets were used in their experiment: two Wikipedia-based (Natural Questions (NQ) and HotpotQA), and two non-Wikipedia-based (Qasper and MultiFieldQA). For the Wikipedia-based datasets, the metric used is Exact Match (EM), since these tasks require short, extractive answers typically under five tokens. For the non-Wikipedia-based datasets, which involve longer, more complex answers, the metric used is token-level F1 score. On NQ, LongRAG achieves an EM of 62.7% using GPT-4o as the reader with 4 retrieved grouped-document units, matching the performance of the fully supervised state-of-the-art model Atlas (64.0% EM) without any fine-tuning. On HotpotQA, it achieves 64.3% EM with 8 retrieved units, approaching the fully supervised COS model (68.2% EM). On Qasper, LongRAG achieves an F1 of 25.9% using a single document-level retrieval unit, outperforming the traditional RAG baseline of 22.6% F1 achieved with 100 short passage-level units. Similarly, on MultiFieldQA-en, it achieves 57.5% F1 with 5 document-level units, compared to 51.3% F1 for 100 short passage units under traditional RAG. The key insight is that increasing retrieval unit size from short passages (100 tokens) to grouped documents (4–6K tokens) reduces the corpus size by up to 30× (from 22M to 600K units for NQ), dramatically improving top-1 answer recall (from 52.2% to 71.7% on NQ) while simultaneously reducing the exposure to hard negatives that confuse the reader.

b) *Context Length and Performance:*

Leng et al. [36] present a comprehensive empirical study evaluating the impact of increasing context length on RAG performance across 20 popular open source and commercial LLMs. Three datasets were used in their experiments: Databricks DocsQA (a domain-specific technical benchmark of 139 queries about the Databricks platform), FinanceBench

(a financial question answering benchmark of 150 queries), and Natural Questions (a standard academic open-domain QA benchmark of 534 queries from the development split). The metric used across all three datasets is answer correctness, evaluated via a calibrated LLM-as-a-judge using GPT-4o, which demonstrated 88.1% agreement with human reviewers. Context length was varied from 2,000 to 128,000 tokens (and up to 2 million tokens for Gemini models), with retrieval performed using OpenAI text-embedding-3-large with 512-token chunks stored in a FAISS index. The results reveal a clear stratification between frontier commercial models and the rest: o1-preview achieves the highest average answer correctness of 0.763, followed by o1-mini (0.731), GPT-4o (0.709), Claude 3.5 Sonnet (0.695), and Claude 3 Opus (0.686), all of which maintain consistent or improving accuracy up to 100k tokens; by contrast, the majority of open source models peak at 16k–32k tokens before degrading, with Llama 3.1 8B dropping from 0.547 at 8k to 0.150 at 128k and DBRX collapsing from 0.539 at 8k to 0.255 at 32k. Notably, Gemini 1.5 Pro and Flash, while lower in absolute accuracy than the top-tier commercial models, uniquely maintain stable performance all the way to 2 million tokens (0.609 and 0.528 respectively), a capability not observed in any other tested model. The key insight is that increasing retrieval context length does not uniformly improve RAG performance, retrieval recall increases monotonically with context length, yet downstream answer correctness does not, implying that models' ability to extract relevant information from long contexts is the binding constraint rather than retrieval coverage itself. Failure mode analysis further reveals that models degrade in qualitatively distinct ways: Claude 3 Sonnet increasingly refuses to answer at longer contexts due to perceived copyright concerns, Gemini 1.5 Pro experiences growing API-level safety filter rejections on the NQ dataset, Mixtral-8x7B produces repeated or random content, and DBRX fails to follow instructions by summarizing retrieved documents rather than answering questions, underscoring that long context degradation is not a single phenomenon but a collection of model-specific failure patterns that must be addressed independently.

c) *KV Cache Optimizations:*

Ge et al. [37] introduces FastGen, an adaptive KV cache compression method that reduces the amount of GPU memory consumed to store key and value vectors for context tokens. The generative inference of LLMs usually implements a KV Cache mechanism for faster generation. However, the memory consumption of KV cache increases rapidly as the model size and generation length increase and that's where KV cache compression methods like FastGen come to be very useful. The method is motivated by the empirical observation that each head follows a specific pattern when choosing which tokens to attend to for context. Some heads attend to special tokens such as beginning-of-sentence tokens or instruction tokens, others focus on punctuation, others on locally recent tokens, and others on the most frequently attended tokens. Rather than applying a uniform compression policy across all attention heads, FastGen conducts a profiling step during prompt encoding to identify the most memory-efficient policy that still recovers at least 95% of the original attention map. A key empirical justification for this approach is that the structural pattern of each attention head remains stable across all subsequent decoding steps, meaning the profile derived from the prompt is sufficient to guide compression throughout the entire generation process. FastGen is evaluated on Llama 1 models at scales from 7B to 65B across four benchmarks covering math (GSM8k), code (HumanEval), open-domain QA (NQ), and reading comprehension (TQA). Results show that FastGen consistently and significantly outperforms fixed non-adaptive compression baselines at any given KV cache budget. Notably, on the 30B model, FastGen at 50% cache compression surpasses all fixed compression methods that compress only 15% of the cache. With a negligible quality loss, FastGen achieves approximately 40% memory reduction for Llama 1-65B, 30% for Llama 1-30B, and around 20% for the 7B and 13B models.

Lu et al. [38] proposes TurboRAG, a redesigned RAG inference paradigm that eliminates the redundant online computation of KV caches by pre-computing and storing document KV caches offline, then retrieving and concatenating them directly at inference time. This solves one of the main limitations of standard RAG, which is the need to reprocess and recompute the entire retrieved context from scratch every time a user submits a query. The waiting time for a response (TTFT) becomes unacceptably long when the retrieved documents are lengthy, making standard RAG unsuitable for any application where response speed matters. TurboRAG utilizes two key observations: cross-attention is very rare as documents primarily attend to their own internal content rather than to each other; and KV caches computed for individual documents can be validly reused if position IDs are correctly rearranged at concatenation time. Based on these two observations, the authors introduced Independent Attention, allowing each document to only attend to its own tokens, and Reordered Positions, which reassigns position IDs at concatenation time to form a proper monotonically increasing sequence across all retrieved chunks. To adapt the pretrained LLM to this new attention and positional regime without accuracy regression, TurboRAG fine-tunes Qwen2-7B using the same Independent Attention mask and Reordered Positions during training, on a dataset of 50% document QA and 50% general tasks. The method is evaluated on the RGB benchmark for multi-noise document QA accuracy and LongBench multi-document QA for both accuracy and TTFT, using GPT-4o and a standard fine-tuned RAG model as baselines. After fine-tuning, TurboRAG-reordered maintains accuracy within 1% of standard RAG on

both benchmarks across all noise levels, and matches GPT-4o performance on both Chinese and English datasets even under high-noise conditions. In terms of inference efficiency, TurboRAG reduces TTFT by an average of 8.6x and up to 9.4x compared to standard RAG, achieved through a 98.46% reduction in online computational load since all document KV cache computation is shifted offline. The speedup grows further with batch size, reaching 32.8x at batch size 8 when KV caches are pre-loaded to GPU memory, because standard RAG's prefill cost scales quadratically with both context length and batch size while TurboRAG's online cost remains nearly flat regardless of how many requests are being served simultaneously.

d) *Tuned Approach:*

SELF-RAG, from Asai et al. [39], introduces a framework where a single language model is trained end-to-end to adaptively retrieve passages on demand and reflect on both the retrieved content and its own generations using special tokens called reflection tokens, rather than indiscriminately retrieving a fixed number of passages as in standard RAG. Six different datasets were used in their experiments across three task categories. The first category is two closed-set tasks including fact verification and multiple-choice scientific reasoning and was evaluated using accuracy. The second category had two short-form open-domain QA tasks and was evaluated using a soft exact match (whether the gold answer is included in the model generation). The last category evaluated using factual precision for biographies, and a combination of correctness,

TABLE I. SUMMARY OF THE DISCUSSED STUDIES IN THE HYBRID RETRIEVAL STRATEGIES FOR LARGE LANGUAGE MODELS SECTION.

Study	Problem Addressed	Core Approach	Key Results
Li et al. *SELF-ROUTE*	LC consistently outperforms RAG but is far more expensive. Yet 63% of queries yield identical results from both, making full LC wasteful for the majority of queries.	Two-step routing: first attempt RAG; if the LLM declares the query unanswerable given the retrieved chunks, escalate to the full LC pipeline. No fine-tuning required.	Achieves near-LC quality while routing the majority of queries to the cheaper RAG pipeline, significantly cutting token usage with only a negligible drop in overall accuracy.
Jiang et al. *LongRAG*	Short retrieval passages (100 tokens) cause low answer recall and expose the reader to many irrelevant chunks that confuse the model.	Group related documents into large retrieval units of 4–6K tokens, reducing corpus size up to 30× and feeding grouped units directly to a long-context LLM reader.	Approaches fully supervised state-of-the-art performance across all four datasets without any fine-tuning, outperforming traditional short-passage RAG baselines on every benchmark.
Lu et al. *TurboRAG*	Standard RAG recomputes KV caches for retrieved documents from scratch on every query, causing high Time-To-First-Token (TTFT) that makes RAG unsuitable for latency-sensitive applications.	Pre-compute and store document KV caches offline. At inference time, assemble the cache by concatenation using Independent Attention (no cross-document attention) and Reordered Positions to preserve correct relative distances. Fine-tune on the new attention regime to prevent accuracy regression.	Reduces TTFT by up to 9.4× with nearly identical answer accuracy to standard RAG, while eliminating almost all online computational overhead. Gains are more at larger batch sizes.
Ge et al. *FastGen*	KV cache memory consumption grows rapidly with model size and sequence length, creating GPU memory pressure that forces expensive offloading and slows inference.	Profile each attention head during prompt encoding to identify its structural pattern (special tokens, punctuation, local context, or frequent tokens), then apply the most memory-efficient compression policy per head. Profile is computed once and remains valid throughout decoding.	Achieves substantial memory reduction across all model sizes with negligible quality loss, consistently outperforming fixed non-adaptive compression methods at every memory budget tested.
Leng et al. *Context Length Study*	It is unclear whether simply increasing retrieval context length improves RAG answer quality, or whether models eventually fail to extract relevant information from very long contexts.	Empirical evaluation of 20 open-source and commercial LLMs across context lengths from 2K to 2M tokens on three QA datasets, using a GPT-4o judge calibrated to 88.1% human agreement.	Frontier commercial models maintain accuracy at very long contexts while most open-source models degrade sharply beyond 16K–32K tokens. More context does not guarantee better answers, as the bottleneck is the model's ability to extract relevant information, not retrieval coverage.
Asai et al. *SELF-RAG*	Standard RAG retrieves a fixed number of passages indiscriminately for every query, regardless of whether retrieval is needed, and does not verify whether retrieved content is actually relevant or used correctly.	Train a single LLM end-to-end to adaptively decide when to retrieve, what to do with retrieved passages, and whether its own outputs are supported, using special "reflection tokens" inserted during training and inference.	A 7B model trained with SELF-RAG matches or exceeds much larger proprietary systems across all benchmarks, demonstrating that retrieval quality and self-reflection matter more than raw model scale.

fluency, and citation precision and recall. On the closed-set tasks, SELF-RAG 7B achieves 72.4% accuracy on PubHealth and 67.3% on ARC-Challenge, outperforming all non-proprietary baselines with and without retrieval. On short-form QA, SELF-RAG 7B achieves 54.9% on PopQA and 66.4% on TriviaQA, surpassing retrieval-augmented ChatGPT on PopQA (50.8%) and competitive with it on TriviaQA. On long-form generation, SELF-RAG 7B achieves a FactScore of 81.2% on biography generation, outperforming retrieval-augmented Llama2-chat (79.9%) and closed-book ChatGPT (71.8%), and on ASQA achieves citation precision of 66.9% and citation recall of 67.8%, outperforming all non-proprietary baselines and even surpassing ChatGPT in citation precision (65.1%).

B. Critical Unresolved Questions

Several important questions remain unanswered across the six studies examined.

First, no current work combines inference-level routing with corpus-level retrieval unit optimization. Li et al. [34] demonstrate that intelligent routing between RAG and LC can recover most of LC's quality advantage at a fraction of the cost, while Jiang et al. [35] show that grouping documents into larger semantic units dramatically improves retrieval recall. Yet these two insights have never been evaluated together. A system that groups documents into 4–6K semantic units offline as in LongRAG, then routes queries adaptively at inference time as in SELF-ROUTE, could address the complementary failure modes of both: poor recall from short passages and unnecessary LC escalation for simple queries.

Second, the relationship between context length and extraction ability identified by Leng et al. [36] has no systematic solution in the current literature. Their study shows that retrieval recall increases monotonically with context length while answer correctness does not, and that failure modes are model-specific rather than universal.Yet no subsequent work proposes a principled method for selecting context length per query or per model tier. An adaptive mechanism , analogous to SELF-ROUTE's query-level routing [34] , that predicts the optimal context length based on query complexity and model capability would directly address this gap.

Third, KV cache efficiency and RAG accuracy have been studied independently but never jointly optimized. Ge et al. [37] show that adaptive per-head cache compression achieves 30–40% memory reduction with negligible quality loss, and Lu et al. [38] show that pre-computing document KV caches offline reduces TTFT by up to 9.4× without sacrificing accuracy. Both papers explicitly note that their methods are orthogonal and composable, yet no work evaluates the combined system or studies how compression interacts with retrieval unit size or routing decisions.

Finally, unified benchmarking across these approaches is sorely needed. The six studies use incompatible evaluation protocols that make direct comparison complicated: Jiang et al.[35] use exact match with alias expansion; Leng et al.[36] use a GPT-4o judge calibrated to 88.1% human agreement; Asai etal. [39] use standard exact match, F1, and FactScore; while Ge et al. [37] and Lu et al. [38] evaluate via GPT-4 pairwise win rate and TTFT latency respectively.

C. Key Insights

Several insights emerge consistently across the six studies. First, the choice between RAG and LC is not binary, intelligent routing between the two recovers most of the quality advantage of LC at a fraction of the cost. Li et al. [34] demonstrate that 63% of queries yield identical results from both approaches, meaning the expensive LC pipeline is only necessary for a minority of hard queries. This finding reframes the design question from "RAG or LC?" to "when does each approach add value?"

Second, model capability is the primary constraint on what retrieval strategy is viable. Leng et al. [36] show a clear stratification: frontier commercial models maintain accuracy up to 100K tokens while most open-source models degrade sharply beyond 16K–32K. This means that the large-unit retrieval approach of LongRAG [35] is only reliable with Tier 1 models, and that context length recommendations cannot be generalized across model tiers.

Third, KV cache optimization is a distinct and separately addressable problem from retrieval strategy. Ge et al. [37] show that the memory cost of long-context processing can be reduced by 30–40% through adaptive per-head compression without meaningful quality loss, while Lu et al. [38] show that the latency cost of repeated document processing can be eliminated almost entirely by shifting KV cache computation offline. Together these results suggest that the computational barriers motivating conservative RAG-only deployments can be well addressed, and that the cost-quality tradeoff documented by Li et al. [34] will continue to shift in favor of LC-based approaches as these efficiency methods mature.

Fourth, both TurboRAG [38] and SELF-RAG [39] demonstrate that targeted fine-tuning produces large gains. TurboRAG fine-tunes Qwen2-7B on a dataset of only 50% document QA and 50% general tasks using the new Independent Attention mask and Reordered Positions, and that was sufficient to fully close the accuracy gap introduced by offline KV cache concatenation, recovering performance to within 1% of standard RAG without any architectural change to the model. SELF-RAG similarly demonstrates that end-to-end training with reflection tokens allows a 7B model to match or exceed much larger proprietary systems across all benchmarks.

XI. INDUSTRY PRODUCTS

A. Retrieval Augmented Generation (RAG) & Hybrid Retrieval Platforms

a) *LangChain:*

LangChain is an open-source framework designed to facilitate the integration of Large Language Models (LLMs) with external knowledge sources, making it a useful industry tool for Retrieval Augmented Generation (RAG) systems. LLMs have two limitations: they can process a limited context window (not infinite), and they possess fixed knowledge from the time of their training. LangChain addresses these limitations by providing modules and components that allows LLMs to retrieve, process, and generate content based on external sources in real time, based on what is needed [40].

RAG Capabilities:

LangChain implements RAG at its core using a retrieval pipeline, enabling LLMs to fetch relevant external knowledge [40]. The pipeline goes as follows:

1) Document Loaders: Documents are loaded by connecting to diverse sources, which can include internal documentation systems.
2) Text Splitters: Content of documents are broken down into smaller retrievable chunks compatible with the LLM's context window.
3) Embedding Models: Content is transformed to vector representation allowing texts to be located efficiently.
4) Vector Stores: Specialized databases for storing and searching embeddings for faster retrieval.
5) Retrievers: Documents are fetches in response to unstructured queries, providing relevant context for generation.

LangChain supports both 2-step RAG, where retrieval is executed before generation, and Agentic RAG, where an LLM agent decides when and how to retrieve external information in real time. These make LangChain suitable for both predictable use cases and more complex applications.

Hybrid Retrieval Architecture:

Other than using a tradition RAG approach, LangChain enables hybrid retrieval strategies, combining the strengths of both 2-step RAG and Agentic RAG. In hybrid, queries can be enhanced or rewritten to improve retrieval, the relevance of the retrieved documents can be validated, and generated answers can pass through tests and checks for accuracy. This makes hybrid systems incorporate multiple sources and maintain higher answer quality. The hybrid approach also aligns with long-context caching and iterative refinement, enabling applications that not only retrieve information but that also maintain context awareness [40].

Overall, LangChain represents a modern ready framework for RAG and hybrid retrieval, making it a compelling choice for both research and industrial applications that use hybrid retrieval systems, especially those requiring long-context reasoning.

b) *LlamaIndex:*

LlamaIndex is a Python-based framework that enables RAG by combining LLMs with structured and unstructured external knowledge sources. Like other RAG implementations, it also addresses the limitations of finite context windows and static information.

RAG Capabilities:

LlamaIndex allows users to construct vector-based indexes over documents, enabling search and retrieval [41]. The main workflow goes as follows:

1) Document Ingestion: LlamaIndex can read documents from local files or cloud storage using loaders, creating structured nodes that represent chunks of content.
2) Vector Embeddings: Dense vectors capturing semantic meaning across text are generated using LLM embeddings.
3) Query Engines: LLM generates context-aware answers by incorporating the retrieval of relevant document notes by the users issuing queries against an index.
4) Integration with LLMs: Retrieved content can be directly given to an LLM to produce responses forming a classic 2-step RAG pipeline.

Hybrid Retrieval Architecture:

Beyond RAG, LlamaIndex supports hybrid retrieval, combining semantic embeddings with sparse vector representations, which focus on keyword matching and details. This adds on the semantic strengths of the dense vectors. This hybrid approach allows the retrieval system to capture both broad semantic relevance and specific keyword matches [41]. Hybrid retrieval in LlamaIndex is implemented using:

1) Sparse and Dense Vector Storage: Documents are indexed simultaneously in both sparse and dense vectors
2) Hybrid Query Fusion: Results from the searches in both vectors are fused via algorithms such as Relative Score Fusion, Reciprocal Rank Fusion, or custom methods. This ensures that the most relevant nodes are sent to the LLM, apart from their vector type.
3) Custom Sparse Models: Allows developers to choose which sparse retrieval model is used to generate sparse vectors for both documents and for user prompts. This allows the system to use a domain-optimized sparse model, meaning that it assigns importance to terms that matter in a specific field depending on what is needed.

This architecture supports long-context caching, where multiple query interactions can use previously retrieved nodes or embeddings to maintain continuity across the session and across multiple sessions and conversations [41].

Representing a flexible RAG and hybrid retrieval platform, LlamaIndex's design makes it suitable for advances hybrid retrieval applications, research on multiple vector fusion, and

long-context reasoning tasks, making it a strong industry example for hybrid and RAG systems.

B. Codebase & Context Intelligence Tools

Managing and grasping large-scale codebases, especially when it comes to a large-scale business, is challenging due to the need to comprehend the architecture and dependencies across repositories and the need to ensure code quality and consistency during development. A lot of AI tools have emerged to solve these challenges, providing various approaches to be able to comprehend codebases, retrieve context, and integrate workflow. Among many leading solutions, Sourcegraph Cody and Qodo demonstrate different approaches to tackle codebases.

a) *Sourcegraph Cody:*

Sourcegraph Cody focuses on codebase understanding through advanced code search, vector embeddings, and RAG. Its architecture enables multi-layered context retrieval, which pulls context from local files, local repositories, and connected remote repositories. The architecture also enables cross-repository analysis, which supports @ mention context retrieval for up to 10 repositories, allowing developers to trace dependencies across services. Cody also supports a large context window, up to 1 million tokens, when paired with Clause Sonnet 4, facilitating complex refactoring. Sourcegraph Cody also supports integration with external systems and platforms such as Jira, Notion, and Google Docs via the Model Context Protocol (MCP) [42].

While Cody tops at code comprehension, its automated review and test generation capabilities are limited. Response may be truncated due to large diffs, and inconsistent IDE features (like JetBrains vs VS Code) can disrupt developer productivity [42].

b) *Qodo:*

Qodo adopts a quality-first approach, emphasizing automated pull request review, test generation, and compliance enforcement. Its proprietary Context Engine combines RAG with agentic reasoning to analyse diffs and enforce organizational rules in real time and generate context aware unit tests. It also provides multi-repository awareness without explicit repository selection, enabling architecture level understanding directly within review workflows. It also integrates deeply with Git platforms and IDEs making it an easier tool to use [42].

Qodo's approach prioritizes code quality and compliance over pure comprehension. Users report slowed performance and usability barriers in large enterprise projects, but it provides transparency for security evaluation due to its open-source PR-Agent [42].

From a hybrid retrieval perspective, Cody provides insight into RAG based vector search for large codebases, while Qodo demonstrates how context aware pruning and structured integration into workflows can improve code quality.

XII. CONCLUSIONS

We have established that model capability and intelligence are no longer the sole limitations for its performance; rather, the amount, structure, and type of context given to the model present a possibly larger issue. Thus, this paper aims to study the recent scholarship on the context management problem and its proposed solutions to avoid information burial under noise and degraded reasoning, which are further amplified in enterprise settings due to the massive internal repositories and high relevance sensitivity.

Retrieval-Augmented Generation (RAG) grounds the model, but in isolation, proves insufficient due to limited recall. Long Context (LC) has the advantage of wider context windows but presents a latency tradeoff. Accordingly, we then explored more hybrid approaches that attempt to combat this tradeoff through varying compromises. For example, SELF-ROUTE demonstrates an optimal strategy that is not binary, but adaptive. Similarly, retrieval unit grouping (LongRAG), KV cache compression (FastGen), and offline precomputation (TurboRAG) illustrate that computational efficiency and reasoning quality must be optimized jointly rather than independently.

This survey also uncovers the deeper structural challenges for context engineering such as positional bias in long contexts, semantic versus task relevance mismatches, summarization drift, memory conflicts, and vulnerability to prompt injection. Such vulnerabilities further emphasize that context engineering cannot be reduced to retrieval alone. Instead, it requires principled pruning mechanisms, controlled injection strategies, ranking policies for memory, and governance-aware mediation layers to ensure reliability in enterprise workflows. Moreover, the nature of enterprise-grade agentic workflows present a more complex environment for the model, and consequently, increased optimization challenges

The main insight that we present is that the future of enterprise AI systems depends on the design of adaptive, context-aware orchestration layers that dynamically regulate retrieval depth, context length, pruning thresholds, routing decisions, memory ranking, and computational cost. No single method (RAG, long-context, pruning, etc.) solves the context management problem in isolation. Therefore, the next logical step is the development of a Context Optimization Engine: a modular system that integrates adaptive retrieval routing, structured context pruning, controlled injection, and efficiency-aware inference optimization under unified evaluation protocols and metrics. Such a system would move beyond static pipelines toward dynamically optimized reasoning environments capable of balancing between accuracy and operational reliability in complex enterprise repositories.

In conclusion, as LLMs continue to grow in scale and capability, the primary distinction for enterprise-grade AI systems

will no longer be model size alone, but the sophistication of the context engineering frameworks that govern what the model sees, how it sees it, and when it sees it.

ACKNOWLEDGEMENT

The authors would like to express their sincere gratitude to our academic supervisors, Dr. Nouri and Dr. Dina Mahmoud, for their invaluable guidance, constant support, and the structural insights provided throughout the development of this literature survey. Their academic mentorship has been fundamental to the progress of this research.

We are also deeply grateful to our industrial sponsor, Valeo, and specifically to Dr. Ibrahim Sobh for providing the technical problem statement, industrial context, and professional mentorship. His expertise in enterprise-grade AI has been essential in aligning this research with current industry trends and requirements.

Finally, we acknowledge the Department of Computer Science and Engineering at The American University in Cairo for providing the academic environment and resources necessary to conduct this work.

REFERENCES

[1] N. F. Liu *et al.*, "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[2] A. Vaswani *et al.*, "Attention is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998–6008.

[3] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020, pp. 1877–1901.

[4] L. Ouyang *et al.*, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[5] Ç. Demiralp and others, "Making LLMs Work for Enterprise Data Tasks," *arXiv preprint arXiv:2407.20256*, 2024.

[6] D. Thangaraju, "Centralized Enterprise Knowledge Base LLMs: A Framework for Efficient and Reusable AI Solutions," *International Journal of Multidisciplinary Research and Growth Evaluation*, vol. 5, no. 5, pp. 1092–1100, 2024.

[7] S. Filice and others, "Generating Q&A Benchmarks for RAG Evaluation in Enterprise Settings," in *Proceedings of the Annual Meeting of the Association for Computational Linguistics (Industry Track)*, 2025.

[8] S. Packowski and others, "Optimizing and Evaluating Enterprise Retrieval-Augmented Generation (RAG): A Content Design Perspective," *arXiv preprint arXiv:2410.12812*, 2024.

[9] A. Alansari and H. Luqman, "Large language models hallucination: A comprehensive survey," *arXiv preprint arXiv:2510.06265v2*, 2025, [Online]. Available: https://arxiv.org/abs/2510.06265

[10] P. Lewis and others, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *arXiv preprint arXiv:2005.11401*, 2020.

[11] Y. Gao and others, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023.

[12] Y. Gan *et al.*, "Navigating the Risks: A Survey of Security, Privacy, and Ethics Threats in LLM-Based Agents," 2024, [Online]. Available: https://arxiv.org/abs/2411.09523

[13] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," technical report, 2024.

[14] European Data Protection Board, "AI Privacy Risks and Mitigations in Large Language Models," technical report, 2025.

[15] C. Li, C. Zhang, H. Zhuge, and others, "AFLOW: Automated Agentic Workflow Optimization via Monte Carlo Tree Search," *arXiv preprint arXiv:2506.14697*, 2025.

[16] F. Kamalov *et al.*, "Evolution of AI in Education: Agentic Workflows," 2025.

[17] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

[18] Y. Hu *et al.*, "QualityFlow: An Agentic Workflow for Program Synthesis Controlled by LLM Quality Checks," *arXiv preprint arXiv:2501.17167*, 2025, [Online]. Available: https://arxiv.org/abs/2501.17167

[19] D. Hendrycks *et al.*, "Measuring Massive Multitask Language Understanding," *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021, [Online]. Available: https://arxiv.org/abs/2009.03300

[20] A. Srivastava *et al.*, "Beyond the Imitation Game: Quantifying and Extrapolating the Capabilities of Language Models," *arXiv preprint arXiv:2206.04615*, 2022, [Online]. Available: https://arxiv.org/abs/2206.04615

[21] W. Zhong *et al.*, "AGIEval: A Human-Centric Benchmark for Evaluating Foundation Models," *arXiv preprint arXiv:2304.06364*, 2023, [Online]. Available: https://arxiv.org/abs/2304.06364

[22] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, Virtual: Curran Associates, Inc., 2020, pp. 9459–9474. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[23] Z. Li, Y. Liu, Y. Su, and N. Collier, "Prompt Compression for Large Language Models: A Survey," *arXiv preprint arXiv:2410.12388v2*, 2024.

[24] Y. Li, "Unlocking Context Constraints of LLMs: Enhancing Context Efficiency of LLMs with Self-Information-Based Content Filtering," *arXiv preprint arXiv:2303.12102*, 2024.

[25] S. Anagnostidis, D. Pavllo, L. Biggio, L. Noci, A. Lucchi, and T. Hofmann, "Dynamic Context Pruning for Efficient and Interpretable Autoregressive Transformers," *arXiv preprint arXiv:2305.15805*, 2023.

[26] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[27] Z. Pan *et al.*, "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression," *arXiv preprint arXiv:2403.12968*, 2024.

[28] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948, doi: 10.1002/j.1538-7305.1948.tb01338.x.

[29] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing Context to Enhance Inference Efficiency of Large Language Models," *arXiv preprint arXiv:2310.06201*, 2024.

[30] C. E. Shannon, "Prediction and Entropy of Printed English," *Bell System Technical Journal*, vol. 30, no. 1, pp. 50–64, 1951, doi: 10.1002/j.1538-7305.1951.tb01366.x.

[31] P. Rajasekaran, E. Dixon, C. Ryan, and J. Hadfield, "Effective context engineering for AI agents." [Online]. Available: https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[32] M. Research, "Efficient AI applications: context engineering and agents." [Online]. Available: https://www.microsoft.com/en-us/research/blog/efficient-ai-applications-context-engineering-and-agents/

[33] A. Arsanjani, "Context-Engineering Challenges & Best-Practices." [Online]. Available: https://dr-arsanjani.medium.com/context-engineering-challenges-best-practices-8e4b5252f94f

[34] Z. Li, C. Li, M. Zhang, Q. Mei, and M. Bendersky, "Retrieval Augmented Generation or Long-Context LLMs? A Comprehensive Study and Hybrid Approach," *arXiv preprint arXiv:2407.16833*, Oct. 2024, doi: 10.48550/arXiv.2407.16833.

[35] Z. Jiang, X. Ma, and W. Chen, "LongRAG: Enhancing Retrieval-Augmented Generation with Long-Context LLMs," *arXiv preprint arXiv:2406.15319*, June 2024, doi: 10.48550/arXiv.2406.15319.

[36] Q. Leng, J. Portes, S. Havens, M. Zaharia, and M. Carbin, "Long Context RAG Performance of Large Language Models," *arXiv preprint arXiv:2411.03538*, Nov. 2024, doi: 10.48550/arXiv.2411.03538.

[37] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, "Model Tells You What to Discard: Adaptive KV Cache Compression for LLMs," in *The Twelfth International Conference on Learning Representations (ICLR)*, Vienna, Austria, May 2024. doi: 10.48550/arXiv.2310.01801.

[38] S. Lu, H. Wang, Y. Rong, Z. Chen, and Y. Tang, "TurboRAG: Accelerating Retrieval-Augmented Generation with Precomputed KV Caches for Chunked Text," *arXiv preprint arXiv:2410.07590*, Oct. 2024, doi: 10.48550/arXiv.2410.07590.

[39] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," in *The Twelfth International Conference on Learning Representations (ICLR)*, Vienna, Austria, May 2024. doi: 10.48550/arXiv.2310.11511.

[40] LangChain Developers, "Retrieval — Docs by LangChain." 2026. [Online]. Available: https://docs.langchain.com/oss/python/langchain/retrieval

[41] LlamaIndex Developers, "Qdrant Hybrid Search — LlamaIndex Python Documentation." 2026. [Online]. Available: https://developers.llamaindex.ai/python/framework/integrations/vector_stores/qdrant_hybrid/

[42] M. Shah, "Sourcegraph Cody vs Qodo (2026): Code Search vs Review Gates." [Online]. Available: https://www.augmentcode.com/tools/sourcegraph-cody-vs-qodo